Question 3: Automate Application Deployment Using Jenkins

Course: DevOps Foundations & Continuous Integration (Lab 1)
Student Name: Akash
Email: ad1646414@gmail.com
Weightage: 5 Marks
Submission Component: Build Triggers, Post-Build Actions, and Containerized Automated Deployment Steps

---

1. Overview of Deployment Automation

Automated deployment ensures that validated code artifacts transition smoothly from integration into a live runtime environment with zero human intervention and zero downtime. This document outlines the configuration of build triggers, deployment orchestration scripts, post-build cleanup, artifact archiving, and runtime health probing.

---

2. Configuring Build Triggers in Jenkins

1. GitHub Push Webhook Trigger

Jenkins listens for GitHub push events via the `GitHub hook trigger for GITScm polling`. When configured within the Declarative Pipeline:

```
triggers {
    githubPush()
    pollSCM('H/5 * * * *')  // Fallback polling every 5 minutes with jitter
}
```

- **Webhook Endpoint:** `http://<jenkins-ip>:8080/github-webhook/`
- **Content-Type:** `application/json`
- **Event:** `Just the push event`

2. SCM Polling Fallback

In restricted enterprise firewalls where inbound public webhooks cannot reach the internal Jenkins master, SCM polling acts as a dependable fallback mechanism (`H/5 * * * *`). The `H` symbol distributes load across controller threads to avoid resource spikes.

---

3. Automated Deployment Steps & Scripting

Automated Deployment Script (`deploy.sh`)

The deployment step invokes an idempotent shell script (`deploy.sh`) that manages container lifecycle states:

```
#!/bin/bash
set -e

APP_NAME="devops-sample-app"
APP_PORT="5000"
HOST_PORT="5000"
IMAGE_NAME="${APP_NAME}:latest"

echo "=== [1/4] Checking existing container instances ==="
if [ "$(docker ps -q -f name=${APP_NAME})" ]; then
    echo "Stopping container ${APP_NAME}..."
    docker stop ${APP_NAME}
fi
if [ "$(docker ps -aq -f name=${APP_NAME})" ]; then
    echo "Removing container ${APP_NAME}..."
    docker rm ${APP_NAME}
fi

echo "=== [2/4] Building production Docker container ==="
docker build -t ${IMAGE_NAME} .

echo "=== [3/4] Running detached container ==="
docker run -d \
  --name ${APP_NAME} \
  --restart unless-stopped \
  -p ${HOST_PORT}:${APP_PORT} \
  -e APP_VERSION="${BUILD_NUMBER:-1.0.0}" \
  -e ENVIRONMENT="Production" \
  ${IMAGE_NAME}

echo "=== [4/4] Automated Health Verification Loop ==="
sleep 3
MAX_RETRIES=10
RETRY=0
while [ $RETRY -lt $MAX_RETRIES ]; do
    STATUS=$(curl -s -o /dev/null -w "%{http_code}" http://localhost:${HOST_PORT}/health || echo "000")
    if [ "$STATUS" -eq 200 ]; then
        echo "SUCCESS: Container is healthy! HTTP status 200 confirmed."
        exit 0
    fi
    echo "Waiting for healthcheck... attempt $((RETRY+1))/$MAX_RETRIES"
    sleep 2
    RETRY=$((RETRY+1))
done
echo "FAILURE: Healthcheck timed out." >&2
exit 1
```

---

4. Post-Build Actions Configuration

Post-build actions guarantee proper cleanup, reporting, and artifact archiving regardless of build success or failure:

```
post {
    always {
        // 1. Publish JUnit test results for trend analytics
        junit allowEmptyResults: true, testResults: 'test-results.xml'

        // 2. Clean workspace to prevent disk consumption on build agents
        cleanWs deleteDirs: true, notFailBuild: true
    }
    success {
        // 3. Archive immutable deployment packages
        archiveArtifacts artifacts: 'dist/*.tar.gz', fingerprint: true
        echo "Deployment Verification Succeeded for Build #${BUILD_NUMBER}"
    }
    failure {
```

```
        // 4. Alert notifications on pipeline breakages
        echo "CRITICAL: Build #${BUILD_NUMBER} failed! Automated rollback triggered."
    }
}
```

---

5. Deployment Verification & Terminal Evidence

Live cURL Probe Outputs

```
HTTP/1.1 200 OK
Content-Type: application/json
Date: Mon, 03 Nov 2026 17:35:00 GMT

{
  "healthy": true,
  "status": "UP",
  "uptime_seconds": 12.4,
  "version": "1.0.1"
}
```

Key Deployment Takeaways

1. **Idempotence:** Running `deploy.sh` repeatedly produces the exact same valid state without errors.
2. **Graceful Teardown:** Cleaning up previous container IDs frees TCP sockets cleanly.
3. **Smoke Testing Guard:** Automated HTTP probing guards against silent runtime crashes.